

BLINKIT SALES ANALYSIS

Project Report

Python-based Retail Sales Analytics using Excel, Pandas, NumPy and Matplotlib

Includes the supplied Blinkit Sales Analysis Dashboard

Prepared from the public GitHub repository: [gagansingh2007/Blinkit-Sales-Analysis](https://github.com/gagansingh2007/Blinkit-Sales-Analysis)

1. Executive Summary

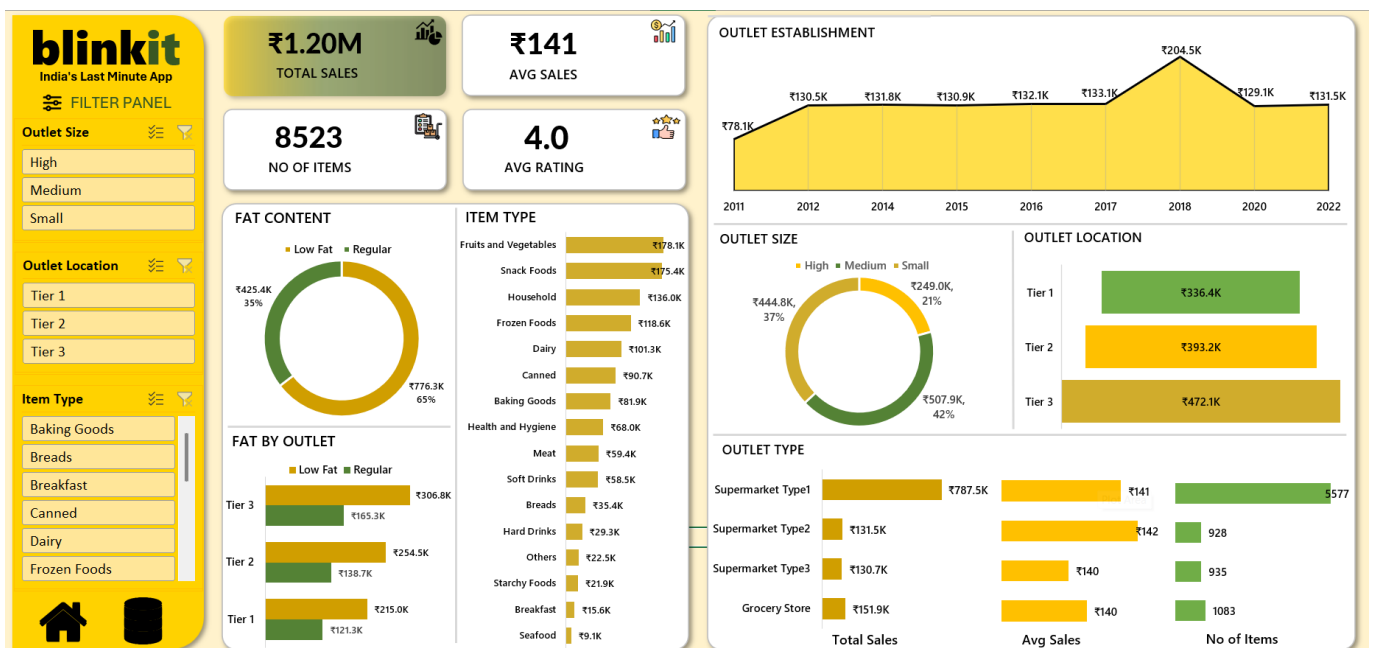
This project analyzes Blinkit grocery sales data at the product-outlet level. The enhanced package combines a reproducible Python analysis pipeline with the supplied dashboard visualization. The dashboard communicates the main business KPIs and sales patterns, while the Python script provides the repeatable data-cleaning and analytical workflow.

2. Business Objective

The objective is to transform raw grocery and outlet data into decision-support information covering sales performance, product categories, outlet characteristics, location tiers, fat-content segments and establishment-year trends.

3. Dashboard Included in the Project

The following dashboard is the supplied visual output for the project. It is included in this report as a full-page reference so that the report, README and repository present the same analytical story.



4. Dashboard Components

- **KPI cards:** Total Sales, Average Sales, Number of Items and Average Rating.
- **Outlet Establishment:** year-wise sales trend with value labels.
- **Fat Content:** Low Fat versus Regular sales contribution.
- **Item Type:** category-wise sales comparison.
- **Outlet Size:** sales distribution across High, Medium and Small outlets.
- **Outlet Location:** Tier 1, Tier 2 and Tier 3 sales comparison.
- **Fat by Outlet:** fat-content sales split within each location tier.
- **Outlet Type:** total sales, average sales and number of items by outlet type.
- **Filter panel:** outlet size, outlet location and item type selections.

5. Data Source

The source repository contains the Excel workbook, requirement document and the dashboard image. The supplied dashboard is therefore treated as part of the project presentation layer, while the Excel workbook remains the source data layer.

6. Data Cleaning & Feature Engineering

- Convert numeric fields safely to numeric types.
- Standardize inconsistent fat-content labels.
- Impute missing item weights using item-type medians followed by the overall median.
- Fill outlet-size gaps using outlet-level information where possible.
- Treat zero item visibility as a data-quality signal and replace it using positive-value median.
- Create outlet age and sales-to-MRP ratio as derived analytical fields.

7. KPI Framework

KPI	Definition
Total Sales	Sum of the sales field.
Average Sales	Mean sales per source record.
Median Sales	Median sales per source record.
Record Count	Number of analyzed rows.
Unique Items	Count of distinct item identifiers, when available.
Unique Outlets	Count of distinct outlet identifiers, when available.
Average Rating	Mean rating when a rating field exists.

8. Analytical Dimensions

The Python pipeline creates grouped summaries for item type, fat content, outlet, outlet size, location type, outlet type and establishment year. It also creates a top-item table when item identifiers are available.

9. Deliverables

- **blinkit_sales_analysis.py** — reproducible end-to-end analysis script.
- **requirements.txt** — Python dependencies.
- **README.md** — GitHub-ready documentation with dashboard section.
- **project_report.pdf** — formal report with the supplied dashboard embedded.
- **Blinkit Analysis Dashboard.png** — supplied dashboard visualization.

10. Reproducibility

Install the dependencies and run the Python script against the repository's Excel workbook. The script generates cleaned data, KPI tables, dimension summaries and analytical charts. The supplied dashboard is retained as the presentation/reference layer.

11. Limitations

- The dashboard is a visualization of the project data; it does not replace the reproducible Python analysis layer.
- Sales relationships should not automatically be interpreted as causal relationships.
- Profitability, delivery performance and customer-level behavior require additional source fields.
- The Python script calculates rating only if the input workbook contains a rating field.

12. Conclusion

With the dashboard embedded, the project now has a clear three-layer structure: the Excel workbook as the data source, the Python script as the reproducible analytical layer, and the dashboard/report as the presentation layer. This makes the project suitable for a portfolio, academic submission or GitHub data-analytics showcase.